

## METODI AVANZATI DI PROGRAMMAZIONE

# Report tecnico

## Alberi di regressione — estensione GUI

Anno accademico 2025/2026

### Gruppo di lavoro

Davide Fornelli • Francesco Cavaliere • Nicola Cialdella

Il report descrive il progetto di Metodi avanzati di programmazione sugli alberi di regressione: versione client/server di base ed estensione grafica Swing.

## Introduzione agli alberi di regressione

Gli alberi di regressione sono modelli di apprendimento supervisionato utilizzati nel machine learning e nel data mining. Prevedono una variabile numerica, detta *target*, a partire dai valori di un insieme di attributi descrittivi.

La costruzione suddivide ricorsivamente gli esempi di addestramento, scegliendo split che riducano la dispersione del target nei sottoinsiemi. Gli attributi discreti distinguono i valori del proprio dominio; quelli continui permettono il confronto con una soglia.

I nodi interni rappresentano le condizioni e i rami i possibili esiti. La costruzione termina quando il sottoinsieme è sufficientemente piccolo oppure lo split non produce più di un ramo. Ogni foglia contiene la media dei target degli esempi associati.

La predizione percorre l'albero dalla radice a una foglia e ne restituisce il valore. Nel progetto l'utente sceglie il percorso rispondendo alle domande presentate dal client.

## Introduzione al progetto

La versione client/server comprende un **server**, responsabile dei dati e dell'albero, e un **client da console**, che gestisce l'interazione con l'utente.

### Server

Il server acquisisce gli esempi da MySQL, costruisce l'albero e lo salva su disco. Può anche ricaricare un albero già appreso. Rimane in ascolto sulla porta indicata, con valore predefinito 8080, e avvia un thread per ogni client. Ogni sessione mantiene dati e albero separati dalle altre connessioni.

### Client da console

Il client riceve indirizzo e porta come argomenti di avvio. Permette di apprendere da una tabella oppure di caricare un albero salvato. Mostra quindi i rami selezionabili e trasmette le scelte al server fino a raggiungere una foglia. Visualizza il risultato e permette di ripetere la predizione sullo stesso albero.

## Estensione GUI

L'estensione aggiunge un'interfaccia desktop realizzata con **Java Swing** e **Java2D**. Consente di apprendere un albero da un file locale oppure da una tabella MySQL attraverso il server MAP, mostrandone nodi, rami, valori predetti e statistiche. Il client da console resta disponibile per la predizione guidata e il caricamento degli archivi.

### Architettura e riuso della base

I sorgenti aggiunti appartengono al package `estensioni.gui` e non modificano il codice applicativo della base. Il training locale riutilizza i package `data` e `tree` dell'applicazione locale `src`, in `project/src/src/`; il training remoto utilizza il server già presente in `project/mapServer/src/`.

I due moduli contengono classi con gli stessi nomi qualificati, ma con modalità di acquisizione differenti: file nel modulo `src`, JDBC nel server. Per questo la GUI include soltanto la base locale nel proprio JAR e comunica con il server tramite socket, senza mescolare le due implementazioni nello stesso classpath.

| Classe                            | Responsabilità                                                                                  |
|-----------------------------------|-------------------------------------------------------------------------------------------------|
| <code>RegressionTreeGUI</code>    | Finestra principale, selezione dell'origine, validazione, avvio del training, stato e dialoghi. |
| <code>RegressionTreeAccess</code> | Adattatore in sola lettura della struttura privata dell'albero del modulo <code>src</code> .    |
| <code>VisualTree</code>           | Conversione dell'albero locale e calcolo delle statistiche della rappresentazione visuale.      |
| <code>VisualNode</code>           | Titolo, metadati, ramo entrante, figli e coordinate di un nodo visuale.                         |
| <code>RegressionTreeCanvas</code> | Disposizione dell'albero, disegno e zoom.                                                       |
| <code>DatabaseTreeClient</code>   | Sessione TCP e ricostruzione dell'albero remoto tramite il protocollo della base.               |
| <code>DatabaseTreeSnapshot</code> | Risultato dell'ispezione di un nodo remoto: domanda di split oppure predizione.                 |
| <code>DatabaseTreeResult</code>   | Albero visuale e dettagli testuali restituiti al termine del training remoto.                   |

### Training locale

`Data` legge il file `.dat`; `RegressionTree` esegue l'induzione ricorsiva già implementata nella base. La soglia di arresto per numero di esempi è il 10% del training set iniziale, calcolato con divisione intera. In alternativa si genera una foglia quando lo split non produce più di un ramo; il valore della foglia è la media del target.

`VisualTree.from()` costruisce una rappresentazione separata, senza cambiare il modello appreso. La base locale non espone radice e figli: `RegressionTreeAccess` legge per riflessione i campi `root` e `childTree`, restituendo una copia dell'array dei figli. Questo conserva l'API originale, ma lega l'adattatore ai nomi e ai tipi dei campi privati.

## Rappresentazione grafica

Entrambe le origini producono un **VisualTree**: il pannello di disegno non deve conoscere né il formato dei file né il protocollo di rete. **RegressionTreeCanvas** misura prima la larghezza dei sotto-alberi e poi dispone ogni padre al centro dello spazio riservato ai propri figli. Nodi e collegamenti vengono disegnati ricorsivamente con antialiasing.

- I nodi interni hanno sfondo nero e mostrano l'attributo di split; le foglie hanno sfondo bianco e valore predetto in rosso.
- Le etichette dei rami riportano uguaglianze discrete oppure confronti continui, conservando gli operatori  $\leq$  e  $>$ .
- I valori predetti sono formattati con il punto decimale e fino a quattro cifre decimali. Questa formattazione non cambia il modello.
- Lo zoom varia dal 50% al 150%, in passi di 10 punti percentuali. Le barre di scorrimento consentono l'esplorazione; **Centra** riporta la vista verso la radice senza modificare lo zoom.

Le statistiche comprendono numero di nodi, foglie e profondità massima, misurata in archi: una sola foglia ha profondità zero. In modalità locale sono mostrati anche numero di esempi, attributi e intervalli degli esempi associati ai nodi. Gli indici si riferiscono ai dati riordinati durante l'apprendimento, non alle righe originarie del file. La modalità database non dispone di questi intervalli e usa metadati descrittivi dell'origine.

## Training remoto e protocollo

La GUI si collega al **server MAP**, non direttamente a MySQL. **DatabaseTreeClient** usa un socket TCP con **ObjectOutputStream** e **ObjectInputStream**. I comandi sono oggetti **Integer**; nomi e messaggi sono stringhe serializzate.

| Codice | Richiesta                                 | Risposta attesa                                                |
|--------|-------------------------------------------|----------------------------------------------------------------|
| 0      | Caricamento della tabella; segue il nome. | <b>Table found!</b> , poi OK.                                  |
| 1      | Apprendimento e salvataggio sul server.   | OK.                                                            |
| 3      | Predizione a partire dalla radice.        | <b>QUERY</b> e domanda; al termine <b>QUERY</b> , OK e valore. |

Il protocollo originale non esporta l'intero albero. Per ricostruirlo, la GUI avvia una predizione per ciascun nodo da ispezionare, ripercorrendo dalla radice gli indici dei rami che lo identificano. Quando incontra lo split cercato, ne conserva la domanda e completa comunque la predizione scegliendo il ramo zero fino a una foglia: in questo modo consuma tutte le risposte e mantiene sincronizzati gli stream. La visita prosegue ricorsivamente sui figli. Un nodo foglia viene ricostruito dal valore numerico ricevuto.

Il codice 2, usato dalla CLI per caricare archivi, non viene inviato dalla GUI. Il server continua a salvare **nomeTabella.dmp** nella propria directory di lavoro; la GUI non trasferisce tale archivio al client.

## Gestione dello stato e degli errori

La finestra viene creata sull'*Event Dispatch Thread* di Swing. Un **CardLayout** alterna i campi per file e database. Le operazioni di training sono eseguite direttamente negli ascoltatori dei pulsanti; durante l'elaborazione viene impostato il cursore di attesa. Al successo, la GUI aggiorna albero e testo, abilita **Dettagli** e pianifica il centraggio.

Prima della connessione vengono controllati host non vuoto, porta numerica nell'intervallo 1–65535 e nome di tabella conforme a `[A-Za-z][A-Za-z0-9_$]*`. Per il file viene verificato che il percorso non sia vuoto; i controlli sul contenuto sono demandati a **Data**.

Un errore preliminare mostra un dialogo senza sostituire l'eventuale risultato precedente. Se invece il caricamento o il training falliscono, l'albero viene rimosso, i dettagli vengono svuotati e disabilitati e la barra mostra **Training non completato**. Il client di rete implementa **Closeable** ed è gestito con *try-with-resources*; la chiusura del socket termina la sessione lato server.

## Limiti dell'implementazione corrente

- Training e comunicazione sono sincroni sul thread Swing: con dati grandi o rete lenta la finestra può non rispondere temporaneamente. Non sono presenti annullamento o avanzamento percentuale.
- La ricostruzione remota ripete percorsi dalla radice. Per un albero di  $N$  nodi e profondità  $h$  richiede fino a un ordine di  $Nh$  scambi lungo i percorsi, oltre all'apprendimento. Non è un trasferimento diretto del modello.
- Il client remoto impone un limite di 10 000 nodi e timeout di 30 secondi per connessione e singole letture. Il timeout non è un limite alla durata complessiva del training.
- La GUI non offre caricamento o salvataggio locale di **.dmp**, esportazione del grafico, modifica dei nodi o predizione interattiva tramite clic. Per le predizioni guidate resta disponibile la CLI.
- I limiti dell'algoritmo base, compresi alcuni casi continui costanti, restano invariati. La visualizzazione non aggiunge correzioni o parametri di apprendimento.
- Il protocollo usa serializzazione Java senza autenticazione o TLS: il servizio è destinato a un ambiente didattico fidato, non all'esposizione diretta su reti non affidabili.

## Compilazione e distribuzione

Dalla cartella principale, con Bash e un JDK installati:

```
bash estensioni/gui/build-gui.sh
java -jar distribution/gui/mapGUI.jar
```

Lo script compila la base locale **src** e le classi della GUI escludendo i test; usa **--release 8**, oppure **-source 8 -target 8** sui JDK meno recenti. La classe principale indicata nel manifest è **estensioni.gui.RegressionTreeGUI**. Il JAR non richiede librerie grafiche esterne né il driver JDBC sul client: **Connector/J** rimane nella distribuzione del server.

## Testing

Le verifiche della versione client/server sono realizzate con JUnit 4 e sono organizzate in due moduli separati, uno per il server e uno per il client. La copertura del codice applicativo viene misurata con JaCoCo. Sono presenti prove delle singole classi e prove di integrazione delle interazioni fra componenti.

I test del server utilizzano un driver JDBC simulato e dataset di prova. Questo consente di riprodurre sia i caricamenti validi sia gli errori di accesso ai dati, senza richiedere un database MySQL attivo. Le prove di comunicazione utilizzano socket locali. Di seguito sono indicati gli ambiti verificati e il trattamento delle classi che non possiedono un test dedicato.

### Package data

- **Data**: vengono verificati l'acquisizione degli esempi, la lettura degli attributi e del target, l'ordinamento e il controllo degli indici. Sono inclusi casi di tabella assente o vuota, schema non valido e guasti simulati durante l'accesso al database.
- **Attribute**: essendo astratta, viene verificata attraverso le istanze delle sottoclassi. I controlli riguardano nome, indice e rappresentazione testuale.
- **DiscreteAttribute**: vengono verificati il dominio dei valori, il numero di valori distinti e l'iterazione.
- **ContinuousAttribute**: non introduce operazioni ulteriori rispetto ad **Attribute**; il costruttore e i membri ereditati sono esercitati dai test degli attributi e dei dati.
- **TrainingDataException**: sono verificati messaggio e causa, oltre alla sua produzione nei percorsi di errore di **Data**.

Percorso dei sorgenti: `project/mapServer/src/data/`.

### Package database

- **DbAccess**: apertura e chiusura della connessione simulata, recupero della connessione e gestione degli errori.
- **TableSchema** e **Column**: lettura dello schema, riconoscimento dei tipi supportati e accesso alle colonne.
- **TableData**: acquisizione delle righe, estrazione dei valori distinti e gestione dei risultati vuoti.
- **Example**: inserimento, lettura e confronto dei valori, rappresentazione testuale e comportamento corrente dell'iteratore.
- **DatabaseConnectionException** ed **EmptySetException**: verifica dei costruttori e delle informazioni conservate.

Le verifiche sono raccolte in **DatabaseClassesTest**. L'enum **QUERY\_TYPE**, annidato in **TableData**, è dichiarato ma non utilizzato nelle operazioni correnti e non ha una prova dedicata.

Percorso dei sorgenti: `project/mapServer/src/database/`.

## Package tree

Le classi astratte `Node` e `SplitNode` sono esercitate mediante `LeafNode`, `DiscreteNode` e `ContinuousNode`. I test controllano le statistiche delle foglie, gli intervalli degli esempi, la formazione dei rami, le soglie e il confronto fra split. Verificano anche le informazioni della classe interna `SplitInfo`.

Per `RegressionTree` vengono controllati l'apprendimento ricorsivo, la struttura risultante, le regole e le domande prodotte. Sono comprese le prove di salvataggio e caricamento, gli archivi non validi e gli errori di scrittura.

Percorso dei sorgenti: `project/mapServer/src/tree/`.

## Package Server

`ServerBootstrapTest` verifica il punto di ingresso `Server` e `MultiServer`: scelta della porta, parametri errati, porta occupata e isolamento di connessioni con handshake non valido.

`ServerOneClientTest` verifica le sessioni attraverso socket locali. Le prove includono acquisizione dei dati, apprendimento, caricamento degli archivi, predizione su più livelli, ripetizione dei nomi non validi e recupero dopo gli errori. Sono controllate anche la separazione dello stato fra client, la chiusura delle risorse e l'eccezione `UnknownValueException`.

Percorso dei sorgenti: `project/mapServer/src/Server/`.

## Package map7Client

`MainTestIntegrationTest` verifica il client da console con un server di prova. Sono esercitati i menu di apprendimento e caricamento, le domande successive, il risultato finale e la ripetizione della predizione. Sono inoltre verificati gli argomenti di avvio, gli errori di rete, le risposte non valide e la fine dell'input da tastiera.

Percorso dei sorgenti: `project/mapClient/src/map7Client/`.

## Package utility

La classe `Keyboard` dispone di test per la lettura di stringhe, parole, caratteri, valori booleani e numeri. Le prove comprendono valori minimi e massimi, input non validi e gestione degli errori di lettura.

Percorso dei sorgenti: `project/mapClient/src/utility/`.

## Risultati dei test

La verifica dei due moduli client/server ha prodotto i seguenti risultati.

| Modulo        | Test      | Fallimenti | Errori   | Saltati  |
|---------------|-----------|------------|----------|----------|
| Server        | 66        | 0          | 0        | 0        |
| Client        | 30        | 0          | 0        | 0        |
| <b>Totale</b> | <b>96</b> | <b>0</b>   | <b>0</b> | <b>0</b> |

La copertura JaCoCo è calcolata sul codice applicativo di ciascun modulo, non sulle classi di test o sul driver simulato.

| Modulo | Copertura delle righe | Copertura dei rami |
|--------|-----------------------|--------------------|
| Server | 98,80%                | 97,88%             |
| Client | 97,78%                | 100,00%            |

Entrambi i moduli superano le soglie di copertura del 90% per righe e rami. Non risultano test disabilitati o saltati nelle due suite eseguite.

Alcune prove descrivono intenzionalmente il comportamento dell'implementazione corrente, compresi casi limite non corretti. Per esempio, viene verificato che `Example.iterator()` restituisca attualmente `null` e che un intervallo continuo costante possa provocare una `NullPointerException`. Il superamento di questi test non equivale alla correzione di tali comportamenti.

Le verifiche JDBC sono svolte in ambiente simulato: i risultati non attestano l'installazione o la connessione a un server MySQL reale. La copertura indica il codice attraversato dai test e non costituisce, da sola, una garanzia di assenza di difetti per qualsiasi input.

## Testing dell'estensione GUI

L'estensione dispone di tre classi JUnit 4 e tre programmi autonomi con asserzioni Java. Sono stati verificati con **OpenJDK 21**, compilando per Java 8, e un display disponibile per le finestre Swing.

- **VisualTreeTest**: conversione di foglie e split discreti e continui, etichette, statistiche, lettura di alberi serializzati, isolamento dell'array dei figli e rifiuto di strutture incomplete. Esercita anche **VisualNode** e **RegressionTreeAccess**.
- **RegressionTreeCanvasTest**: stato vuoto, dimensioni, disposizione dei figli, rendering su immagine, abbreviazione delle etichette e limiti dello zoom, inclusi valori infiniti e NaN.
- **RegressionTreeGUITest**: stato iniziale, training locale, dettagli, file assente o percorso vuoto, centraggio, zoom, chiusura del selettore file, stile dei pulsanti e avvio della finestra.

| Classe JUnit             | Test      | Fall.    | Errori   | Saltati  |
|--------------------------|-----------|----------|----------|----------|
| VisualTreeTest           | 9         | 0        | 0        | 0        |
| RegressionTreeCanvasTest | 4         | 0        | 0        | 0        |
| RegressionTreeGUITest    | 9         | 0        | 0        | 0        |
| <b>Totale GUI</b>        | <b>22</b> | <b>0</b> | <b>0</b> | <b>0</b> |

I programmi autonomi completano i controlli: **DatabaseTreeClientTest** verifica il protocollo con un server simulato; **VisualTreeStatisticsTest** controlla le statistiche di **prova.dat**; **DatabaseTreeIntegrationTest** usa la vera classe **ServerOneClient** con il solo driver JDBC simulato. Quest'ultima prova controlla alberi multilivello, una singola foglia, salvataggio, tabella assente e fallimento della scrittura. Un class loader separato isola le classi **data/tree** del server da quelle del modulo **src**.

**Esito:** 22 test JUnit superati e tre programmi autonomi completati con asserzioni abilitate. Con la base, i test JUnit sono 118, senza fallimenti, errori o test saltati; i programmi autonomi non si sommano al conteggio JUnit.

## Riproducibilità e ambito dei risultati

Dalla cartella principale si possono rieseguire le suite della base e i tre programmi autonomi della GUI con:

```
mvn -f tests/server/pom.xml verify
mvn -f tests/client/pom.xml verify
bash estensioni/gui/build-gui.sh --check
```

Le classi JUnit in **tests/estensioni/gui/** non sono collegate a Maven né eseguite da **--check**. Per avviarle con **JUnitCore**, seguire **docs/README.md**. Senza display i nove test della finestra vengono saltati per assenza JUnit.

Non è stata misurata una copertura JaCoCo della GUI: le percentuali della base non si estendono a questo package. Le prove remote non certificano una connessione a MySQL reale; non sono prove di carico o di usabilità e non coprono ogni errore di rete o ogni validazione dei campi database.